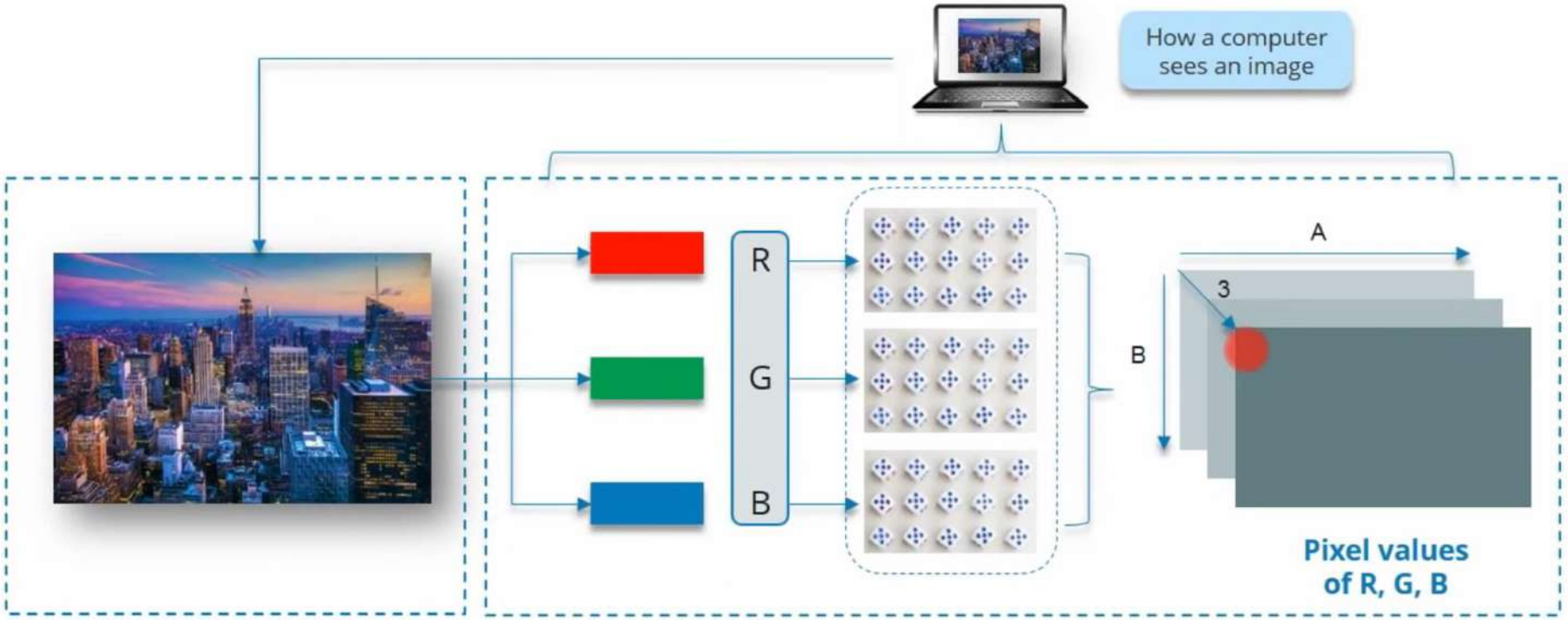
CNN

How a Computer Reads an Image



Convolutional Neural Network (CNN) | Convolutional Neural Networks With TensorFlow | Edureka

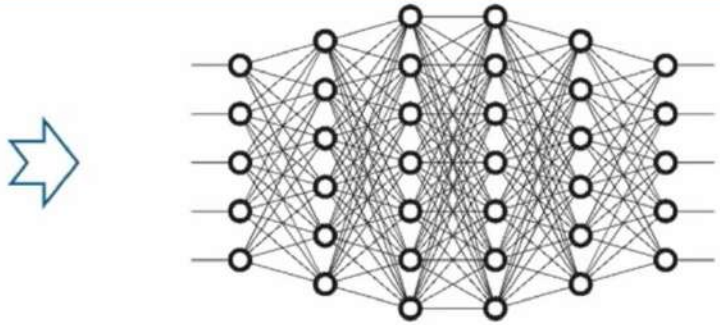
	label	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixel8	pixel9	...	pixel775	pixel77
0	3	107	118	127	134	139	143	146	150	153	...	207	20
1	6	155	157	156	156	156	157	156	158	158	...	69	14
2	2	187	188	188	187	187	186	187	188	187	...	202	20
3	2	211	211	212	212	211	210	211	210	210	...	235	23
4	13	164	167	170	172	176	179	180	184	185	...	92	10

5 rows × 785 columns

https://medium.com/@gryangalario/image-classification-using-logistic-regression-on-the-american-sign-language-mnist-9c6522242ddf

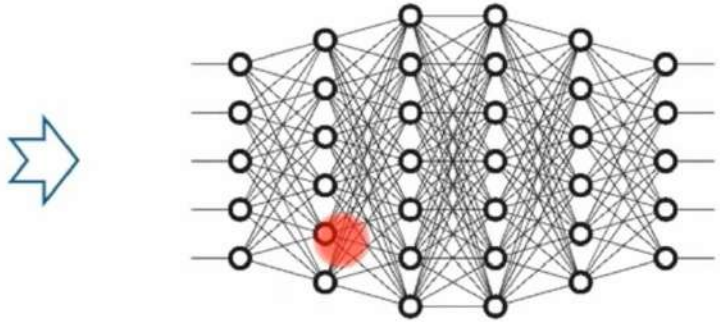
Why Not Fully Connected Networks

Image with 28 x 28 x 3 pixels



Number of weights in the first hidden layer will be 2352

Image with 200 x 200 x 3 pixels



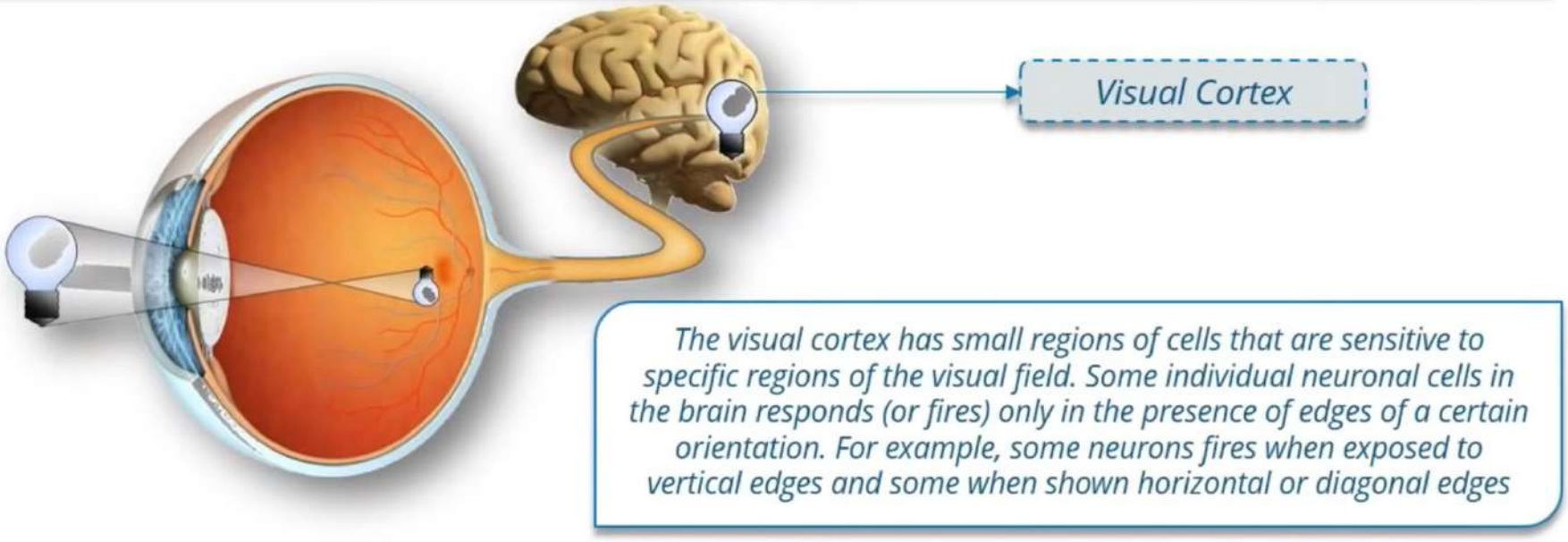
Number of weights in the first hidden layer will be 120,000

Convolutional Neural Network (CNN) | Convolutional Neural Networks With TensorFlow | Edureka

Convolutional Neural networks

What is Convolutional Neural Network?

***Convolutional Neural Network** (**CNN**, or **ConvNet**)* is a type of feed-forward artificial neural network in which the connectivity pattern between its neurons is inspired by the organization of the animal visual cortex.



The visual cortex has small regions of cells that are sensitive to specific regions of the visual field. Some individual neuronal cells in the brain responds (or fires) only in the presence of edges of a certain orientation. For example, some neurons fires when exposed to vertical edges and some when shown horizontal or diagonal edges

Convolutional Neural Network (CNN) | Convolutional Neural Networks With TensorFlow | Edureka

CNN

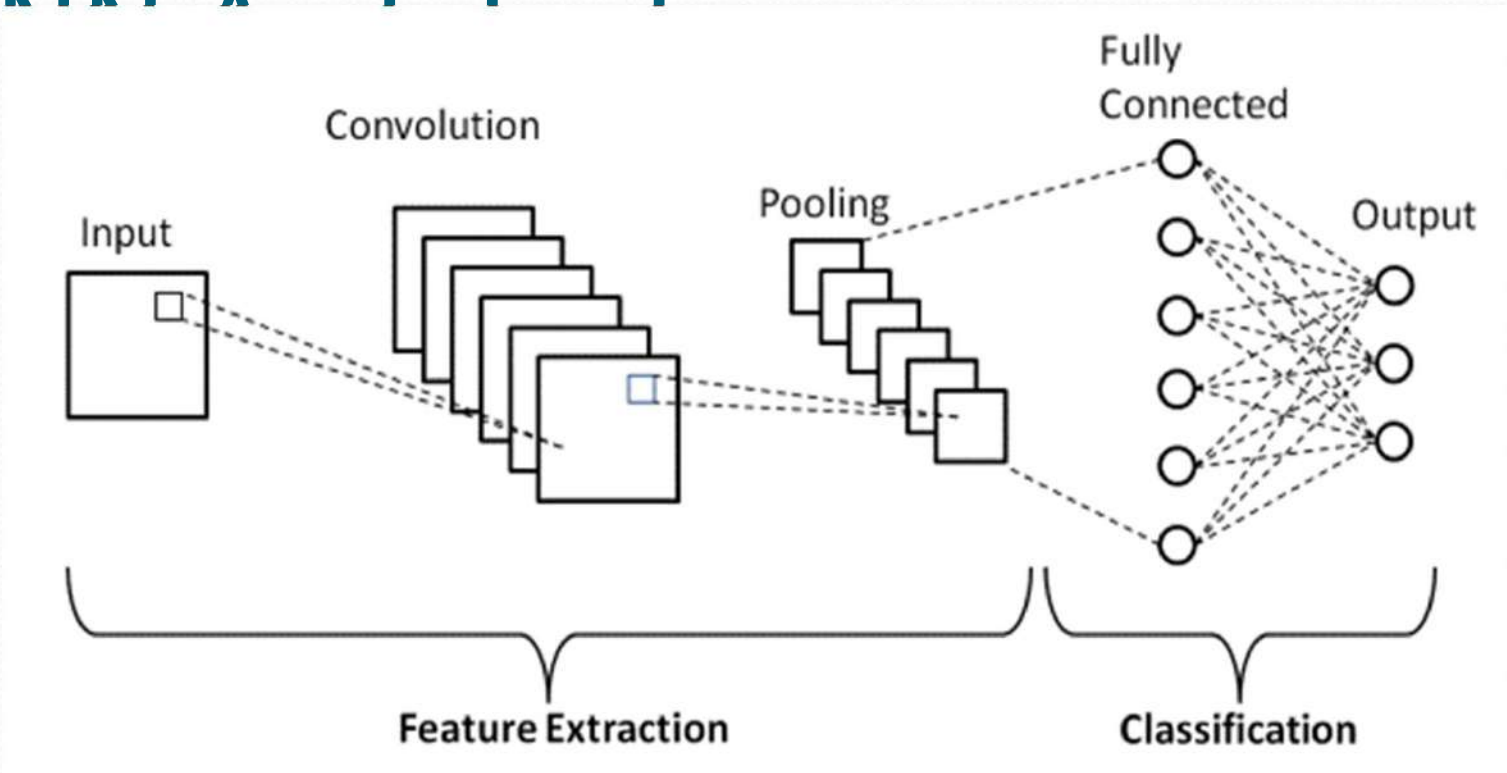
- Convolutional neural networks- feed forward neural network generally used to analyse visual images by processing data with grid-like topology.

- Yann LeCun built the first CNN network called LeNET in 1988.

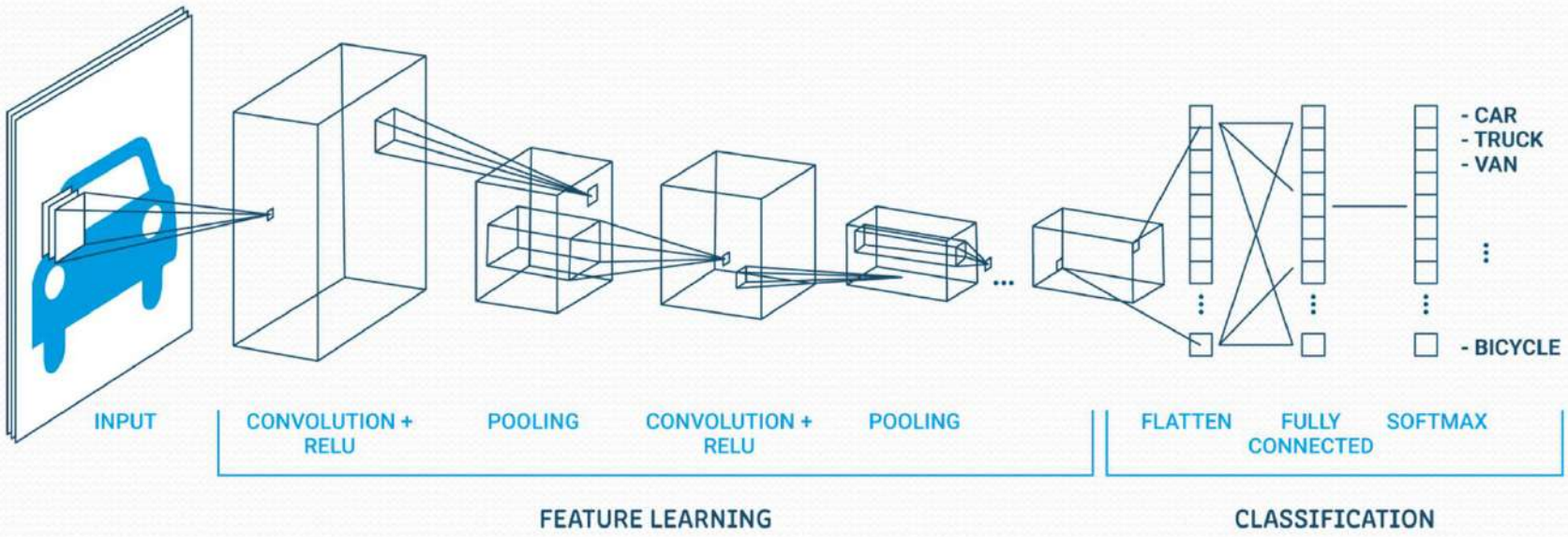
CNN - Applications

- Facial recognition in social media
- Object detection in self driving cars
- Disease detection using visual imagery in healthcare

C



Source: https://medium.com/techiepedia/binary-image-classifier-cnn-using-tensorflow-a3f5d6746697



https://nanonets.com/blog/machine-learning-image-processing/

CNN layers

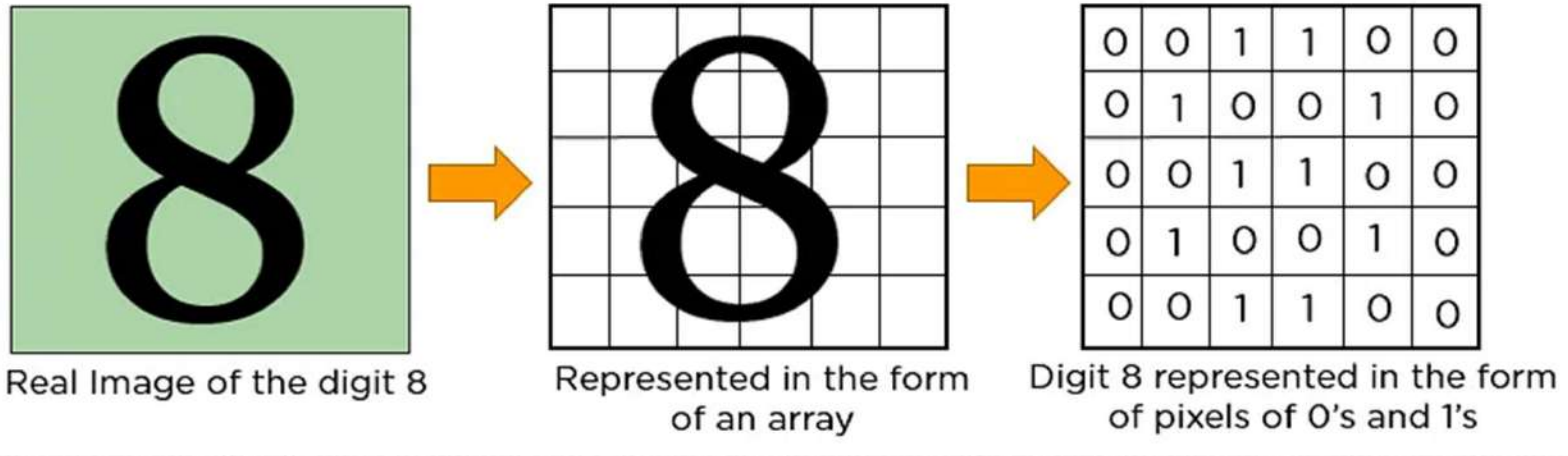
- Input layer
- Hidden layers
  - Convolution layer
  - ReLu layer
  - Pooling layer
  - Fully connected layer
- Output layer

Input layer

- In CNN, every input be it image or text, should be represented as integers.

- Eg: An image is represented in the form of an array of pixel values.

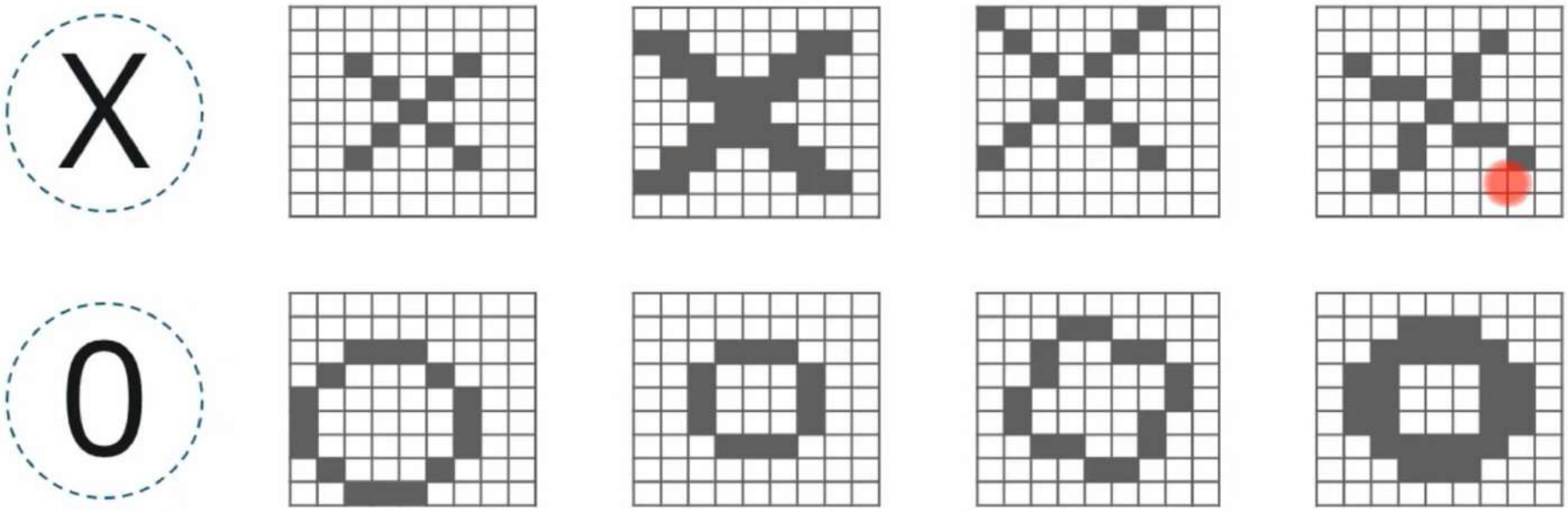
Input layer



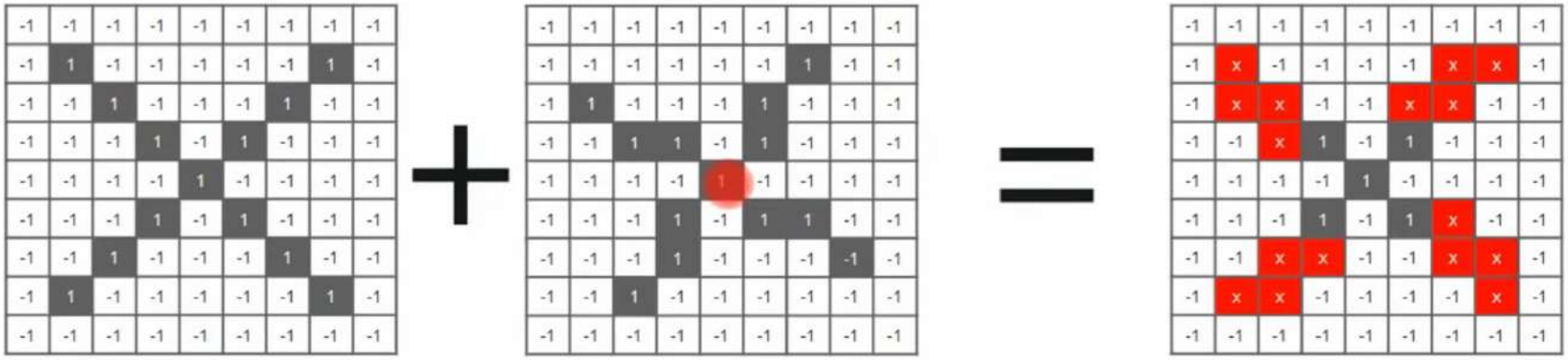
Source: https://www.simplilearn.com/tutorials/deep-learning-tutorial/convolutional-neural-network

Trickier Case

Here, we will have some problems, because X and O images won't always have the same images.
There can be certain deformations. Consider the diagrams shown below:



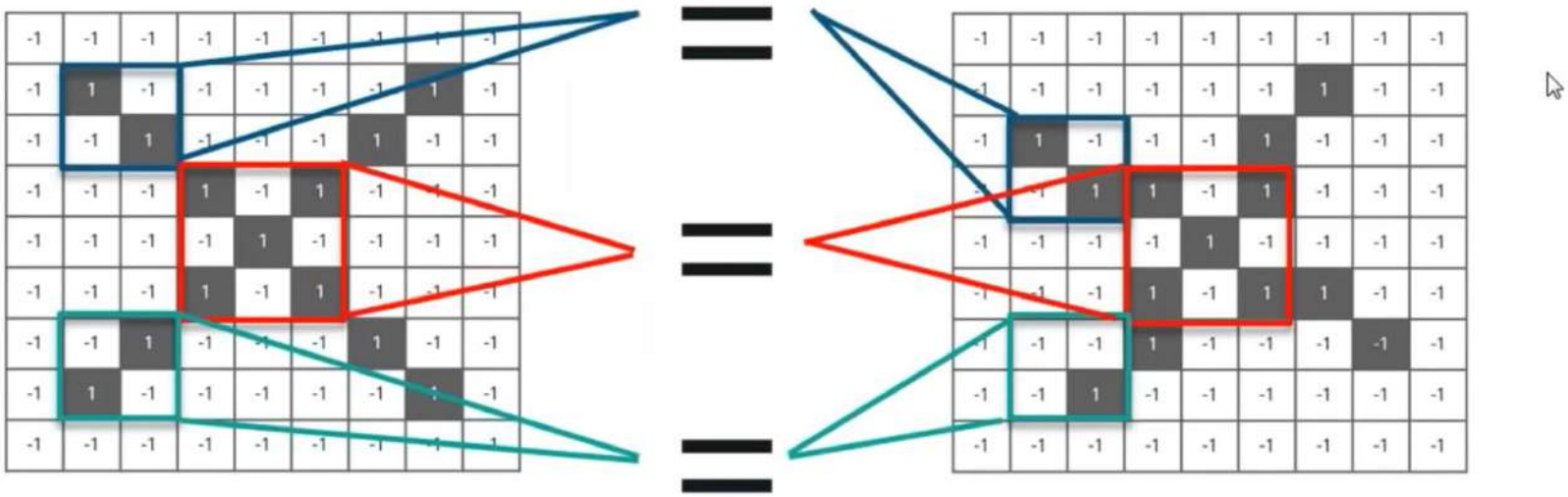
Using normal techniques, computers compare these images as:



Convolutional Neural Network (CNN) | Convolutional Neural Networks With TensorFlow | Edureka

How CNN Works?

CNN compares the images piece by piece. The pieces that it looks for are called features.
By finding rough feature matches, in roughly the same position in two images, CNN gets a lot better at seeing similarity than whole-image matching schemes.



How CNN Works?

We will be taking three features or filters, as shown below:

1	-1	-1
-1	1	-1
-1	-1	1

1	-1	1
-1	1	-1
1	-1	1

-1	-1	1
-1	1	-1
1	-1	-1

How CNN Works?

1	-1	-1
-1	1	-1
-1	-1	1

1	-1	1
-1	1	-1
1	-1	1

-1	-1	1
-1	1	-1
1	-1	-1

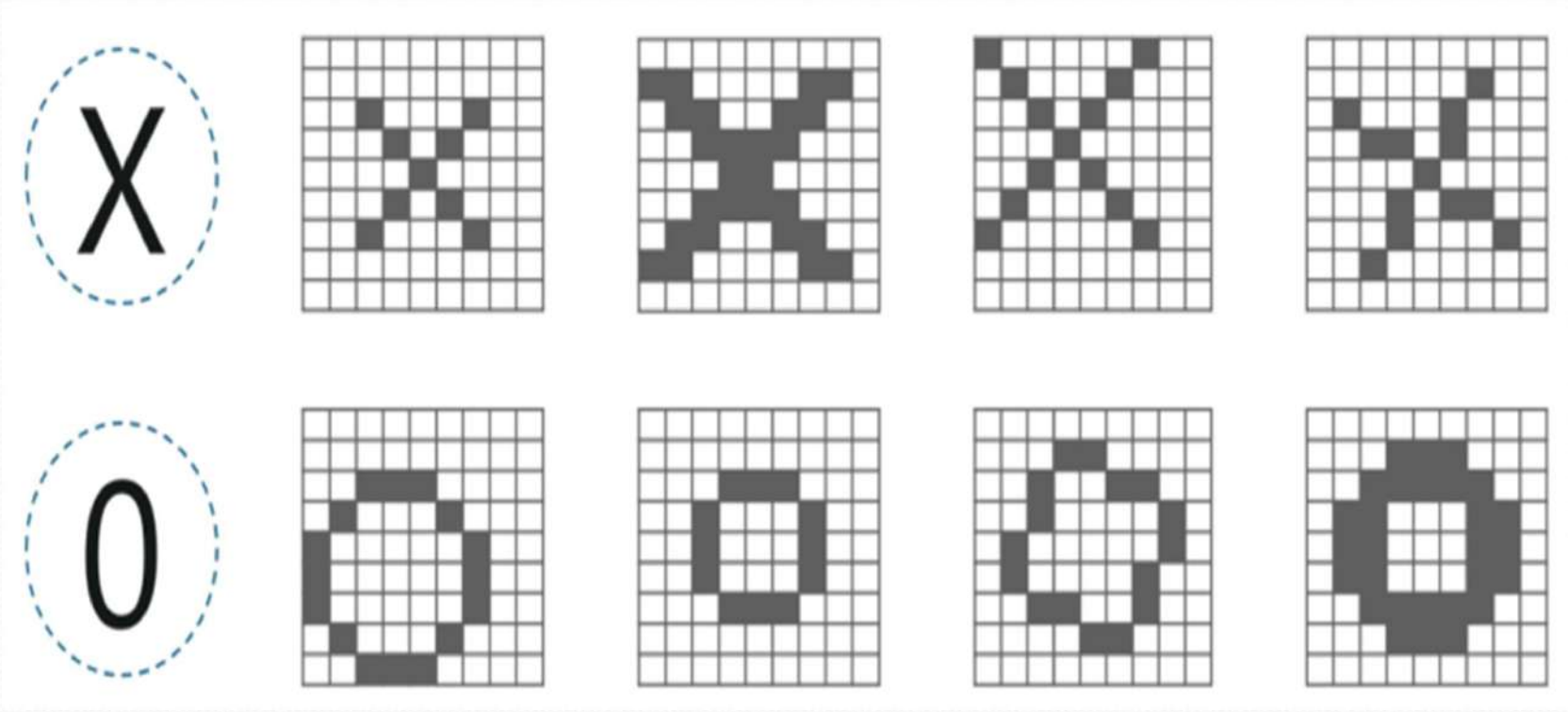
These are small pieces of the bigger image. We choose a feature and put it on the input image, if it matches then the image is classified correctly.

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

Convolution operation

- The basis of CNN is the **Convolution.**
- In CNN, the convolution operation is applied to the input data by using a filter.
- **Input data + filter = feature map**
- Filter is also called “kernel” or “feature detector”.
- Feature map is also called “activation map”.
- If your input data is of size ‘m’ and filter is of size ‘n’, then feature map will be of size ‘**m-n+1**’

Input

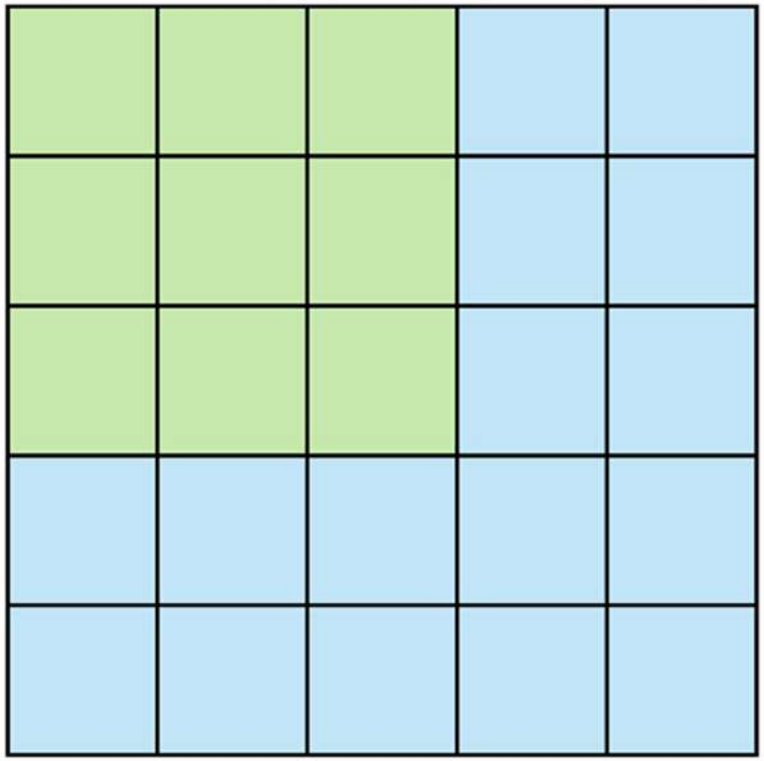


Input

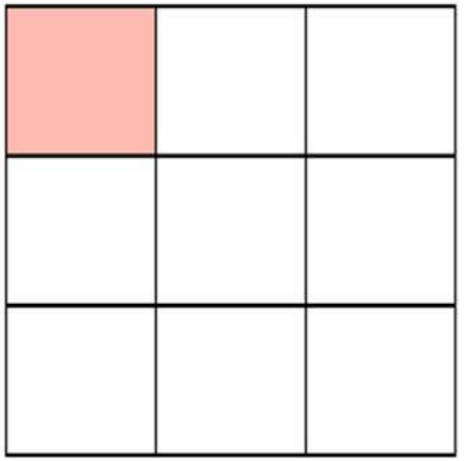
- the **white** pixels are said to be **-1** while the **black** ones are **1**

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

Convolution Layer

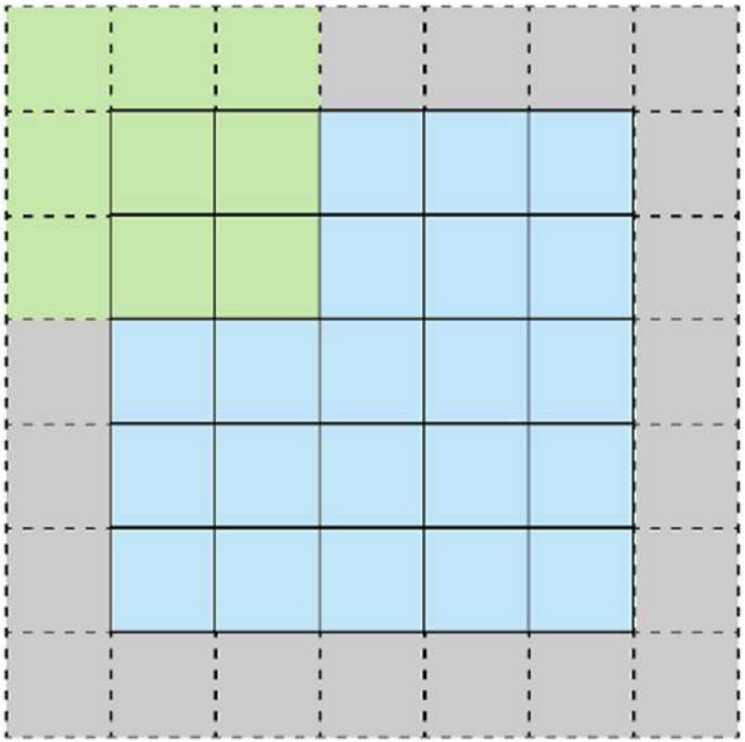


Stride 1

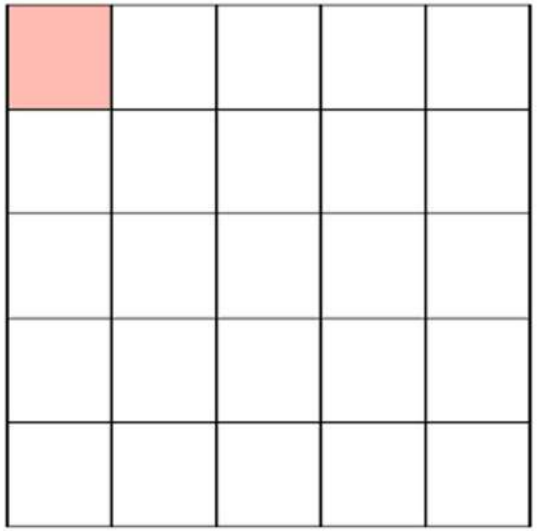


Feature Map

https://towardsdatascience.com/covolutional-neural-network-cb0883dd6529#:~:text=Examples%20of%20CNN%20in%20computer,i.e%2C%20weights%2C%20biases%20etc.



Stride 1 with Padding



Feature Map

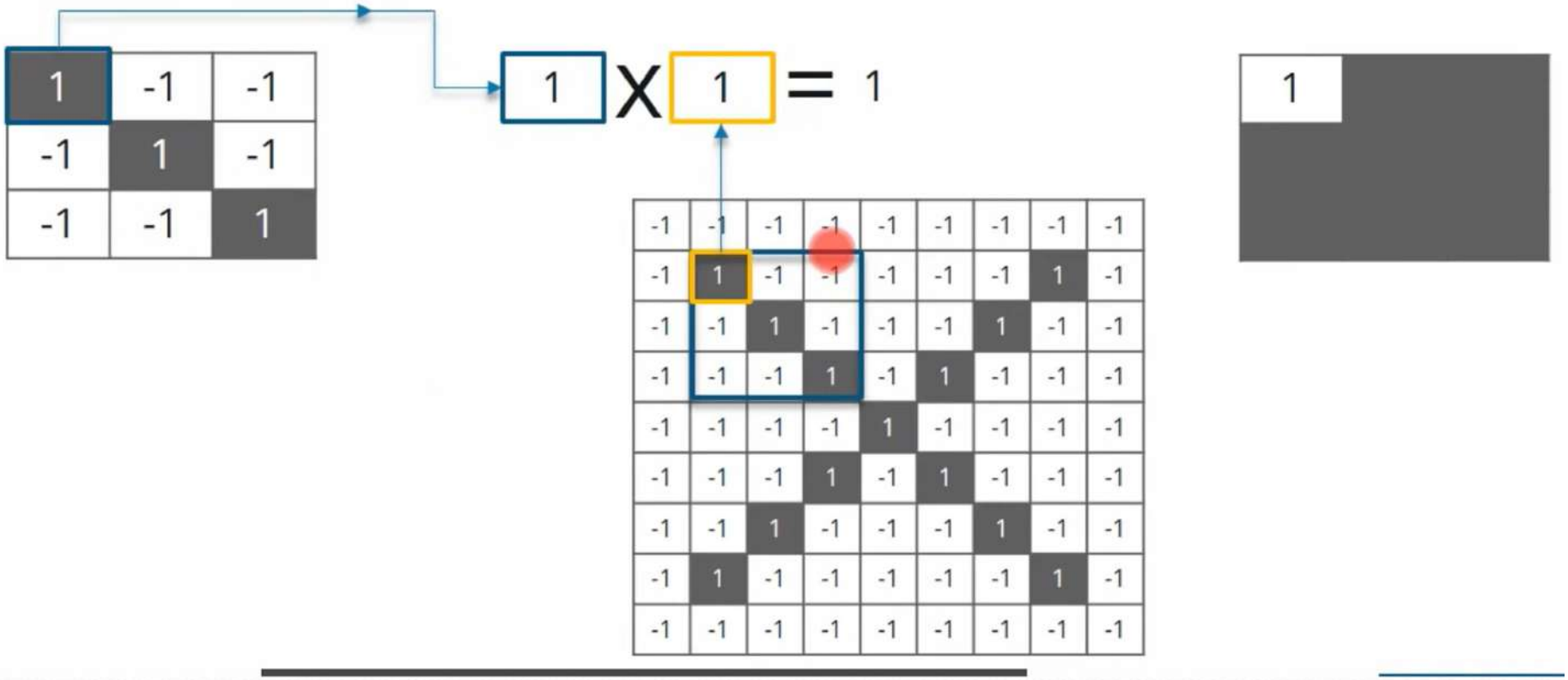
https://towardsdatascience.com/covolutional-neural-network-cb0883dd6529#:~:text=Examples%20of%20CNN%20in%20computer,i.e%2C%20weights%2C%20biases%20etc.

Convolution

- **Line up** the feature and the image
- **Multiply** each **image** pixel by corresponding **feature** pixel
- **Add** the values and find the **sum**
- **Divide** the sum by the **total** number of pixels in the **feature**

Note: Stride denotes how many steps we are moving in each steps in convolution. By default it is one.

Multiplying the Corresponding Pixel Values



Adding and Dividing by the Total Number of Pixels

1	-1	-1
-1	1	-1
-1	-1	1

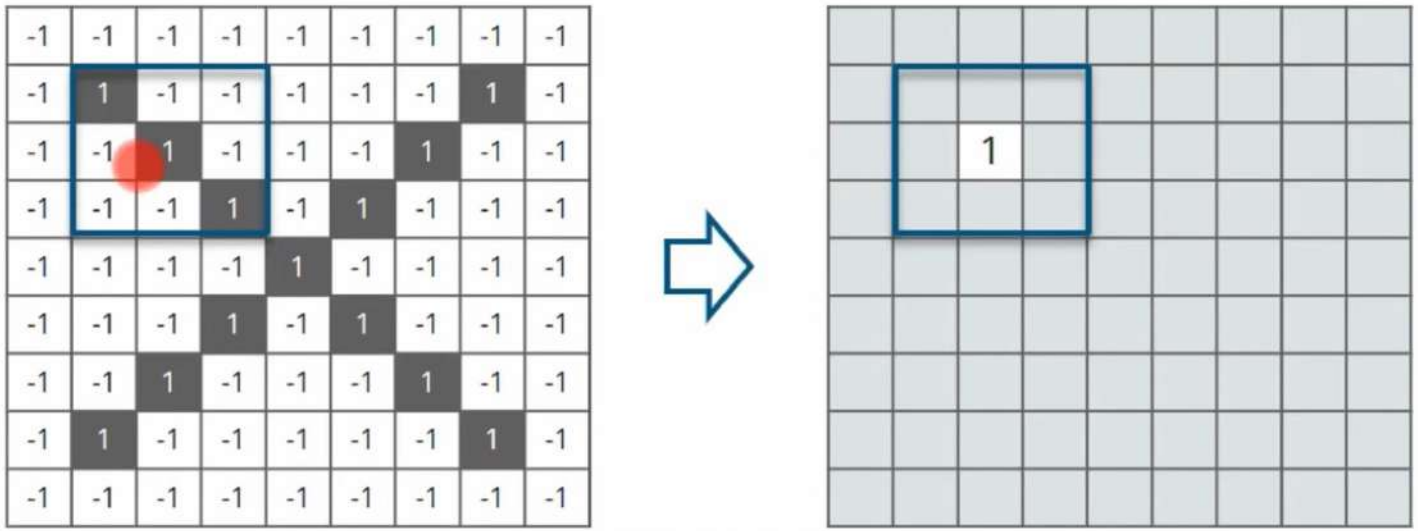
$$\frac{1+1+1+1+1+1+1+1+1}{9} = 1$$

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

1	1	1
1	1	1
1	1	1

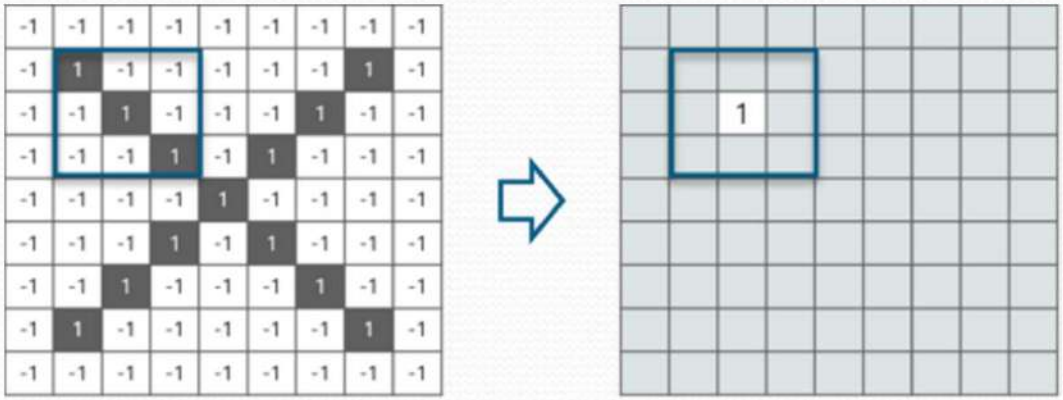
Creating a Map to Put the Value of the Filter

Now to keep track of where that feature was, we create a map and put the value of the filter at that place



Convolution

Convolution process:



Feature map obtained is:

0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.0	-0.11	0.33	-0.11	0.11	-0.11
0.11	-0.11	1.0	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77

Sliding the Filter Throughout the Image

Now, using the same feature and move it to another location and perform the filtering again.

1	-1	-1
-1	1	-1
-1	-1	1

$$\frac{1+1-1+1+1+1-1+1+1}{9} = .55$$

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

1	1	-1
1	1	1
-1	1	1

Convolution Layer Output

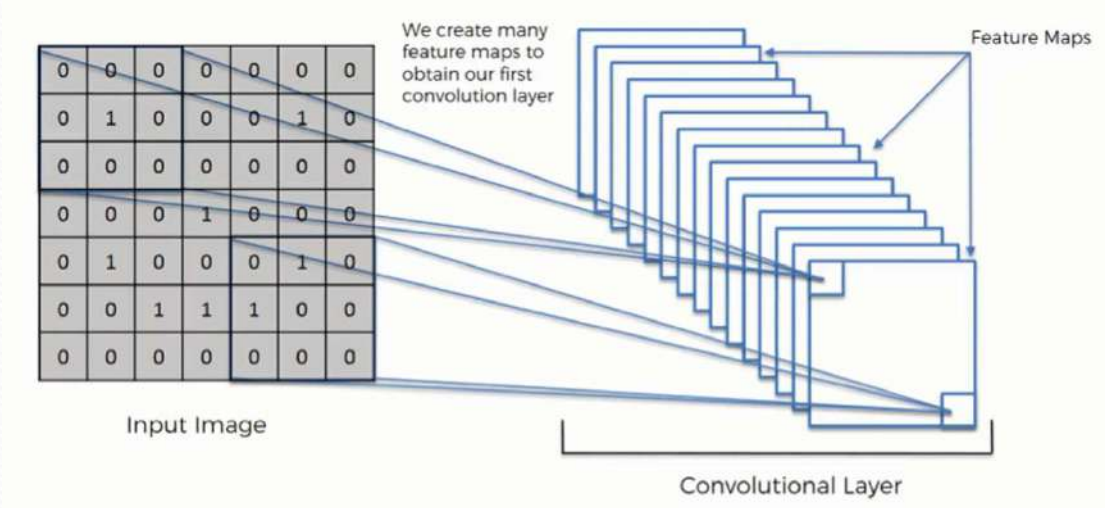
Similarly, we will move the feature to every other positions of the image and will see how the feature matches that area. Finally we will get an output as:

0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.0	-0.11	0.33	-0.11	0.11	-0.11
0.11	-0.11	1.0	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77

What is next..

- we will perform the same convolution with every other filter to get the convolution of that filter.
- All the feature maps thus obtained constitute our convolutional layer.
- CNN uses multiple feature detectors to develop multiple feature maps which becomes the "convolutional layer".

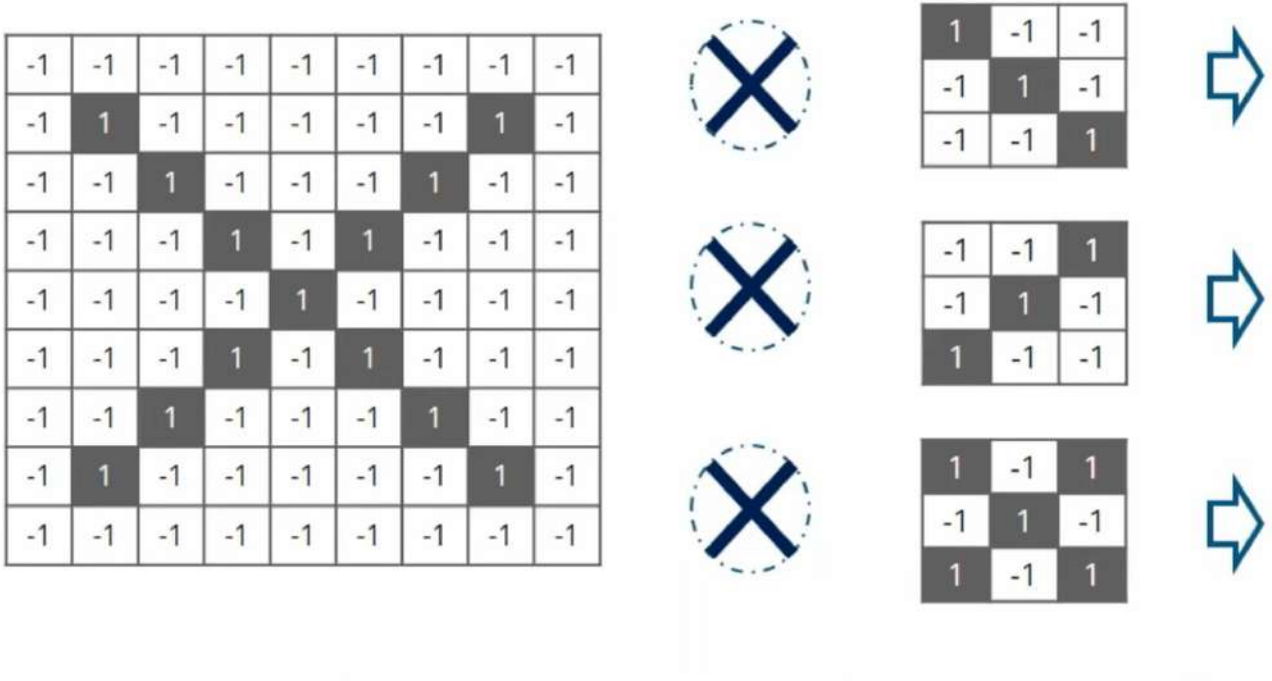
Convolutional layer



Source: https://www.superdatascience.com/blogs/the-ultimate-guide-to-convolutional-neural-networks-cnn

Convolution Layer Output

Similarly, we will perform the same convolution with every other filters



0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.0	-0.11	0.33	-0.11	0.11	-0.11
0.11	-0.11	1.0	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77

0.33	-0.55	0.11	-0.11	0.11	-0.55	0.33
-0.55	0.55	-0.55	0.33	-0.55	0.55	-0.55
0.11	-0.55	0.55	-0.11	0.55	-0.55	0.11
-0.11	0.33	-0.77	1.00	-0.77	0.33	-0.11
0.11	-0.55	0.55	-0.77	0.55	-0.55	0.11
-0.55	0.55	-0.55	0.33	-0.55	0.55	-0.55
0.33	-0.55	0.11	-0.11	0.11	-0.55	0.33

0.33	-0.11	0.55	0.33	0.11	-0.11	0.77
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.11	-0.11	1.00	-0.33	0.11	-0.11	0.55
-0.11	1.00	-0.11	0.33	-0.11	0.11	-0.11
0.77	-0.11	0.11	0.33	0.55	-0.11	0.33

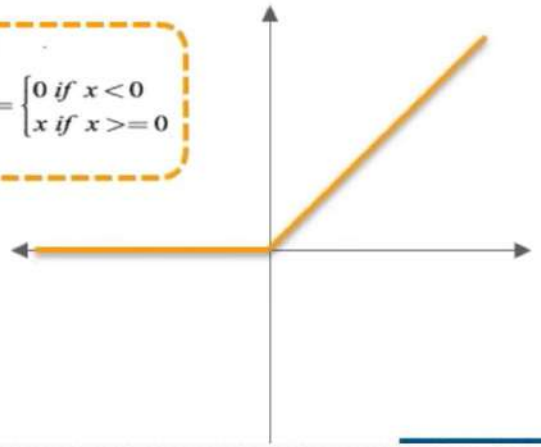
ReLU Layer

- ✓ In this layer we remove every negative values from the filtered images and replaces it with zero's
- ✓ This is done to avoid the values from summing up to zero

Rectified Linear Unit (ReLU) transform function only activates a node if the input is above a certain quantity, while the input is below zero, the output is zero, but when the input rises above a certain threshold, it has a linear relationship with the dependent variable

x	*f(x)=x*	F(x)
-3	*f(-3) = 0*	0
-5	*f(-5) = 0*	0
3	*f(3) = 3*	3
5	*f(5) = 5*	5

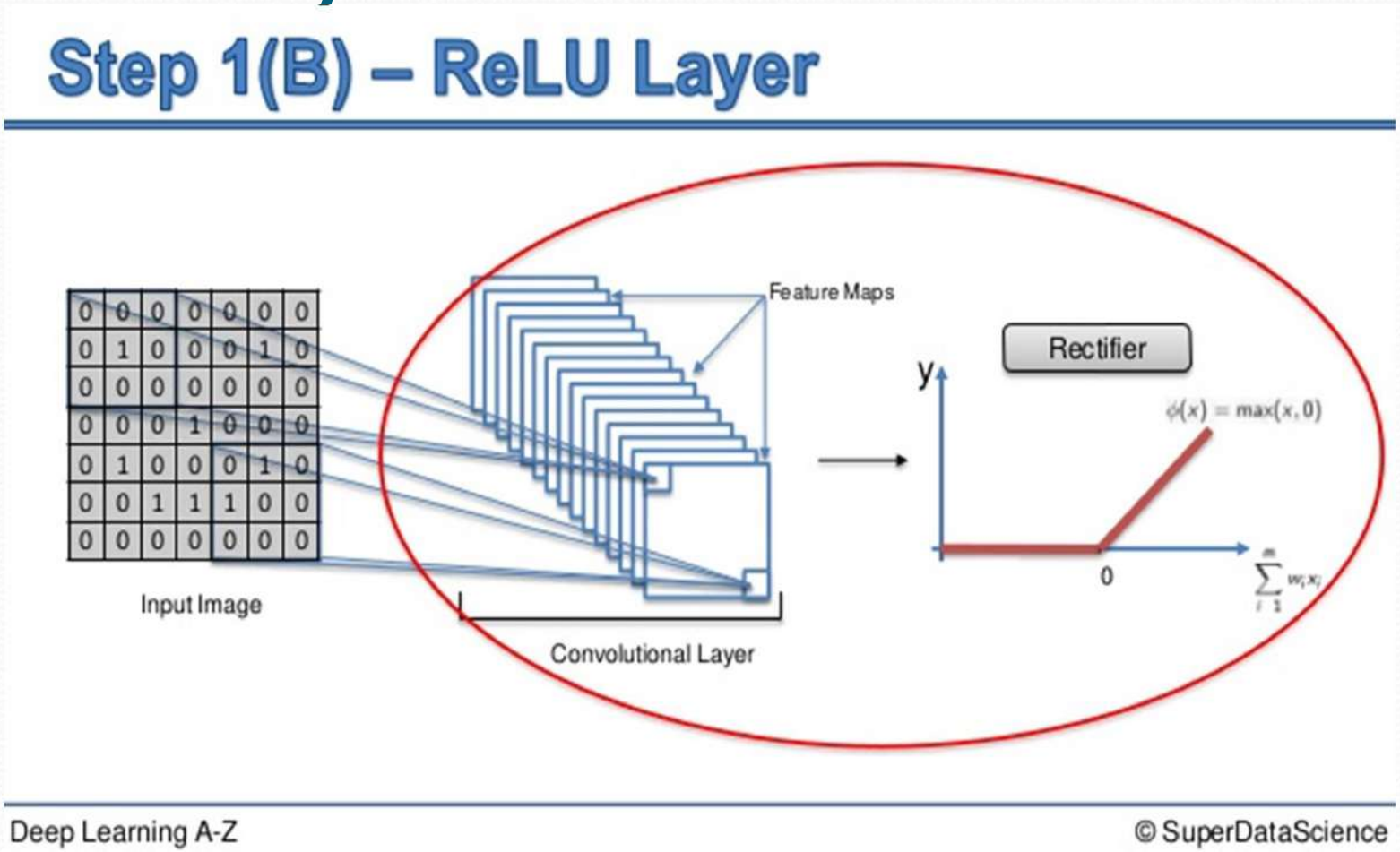
$$f(x) = \begin{cases} 0 \text{ if } x < 0 \\ x \text{ if } x >= 0 \end{cases}$$



ReLU layer

- ReLU – Rectified linear unit
- Performs element –wise operation
- It sets negative pixels to 0.

ReLU layer

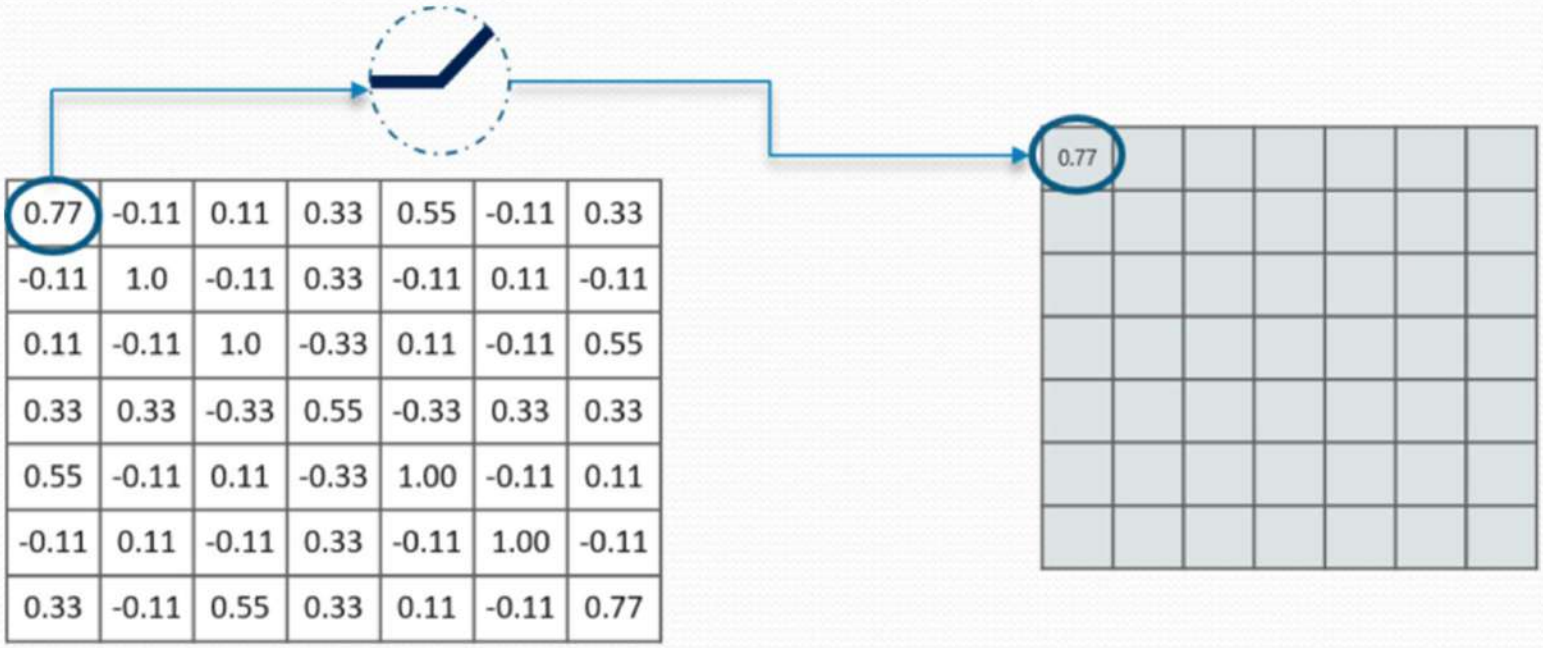

Step 1(B) – ReLU Layer
Feature Maps
Rectifier
y
$\phi(x) = \max(x, 0)$
0
$\sum_{i=1}^{m} w_i x_i$
Input Image
Convolutional Layer
Deep Learning A-Z
© SuperDataScience

Convolution- points to note

- Reduce the size of input image
- Lose of information
- Retain only integral part of the input

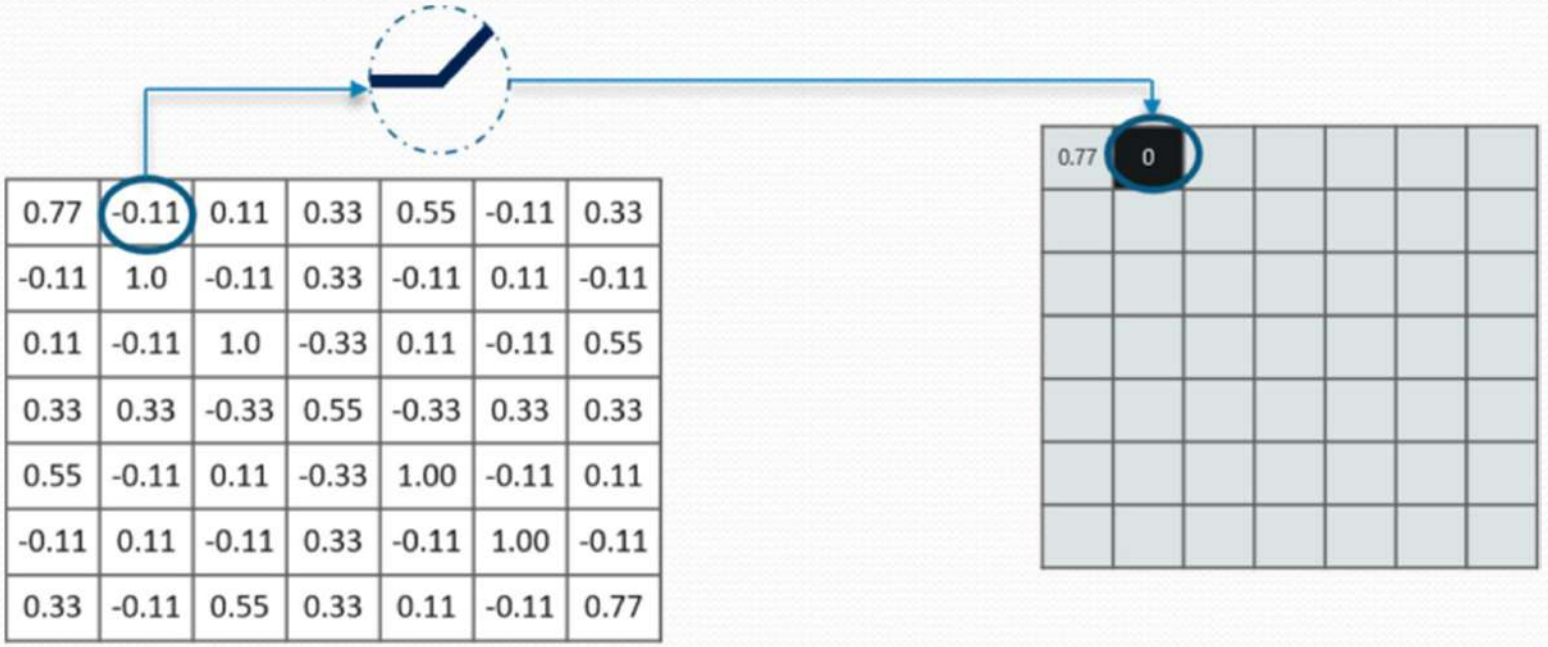
ReLU layer

Postive values are retained as such



ReLU layer

Negative values are converted to zero



0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.0	-0.11	0.33	-0.11	0.11	-0.11
0.11	-0.11	1.0	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77

ReLU layer

After application of ReLU we get a feature map like this.

0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.0	-0.11	0.33	-0.11	0.11	-0.11
0.11	-0.11	1.0	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77

0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	1.77

Do the same for other feature maps also !!!

Pooling Layer

In this layer we shrink the image stack into a smaller size

Steps:

1. Pick a window size (usually 2 or 3).
2. Pick a stride (usually 2).
3. Walk your window across your filtered images.
4. From each window, take the maximum value.

Symbol:

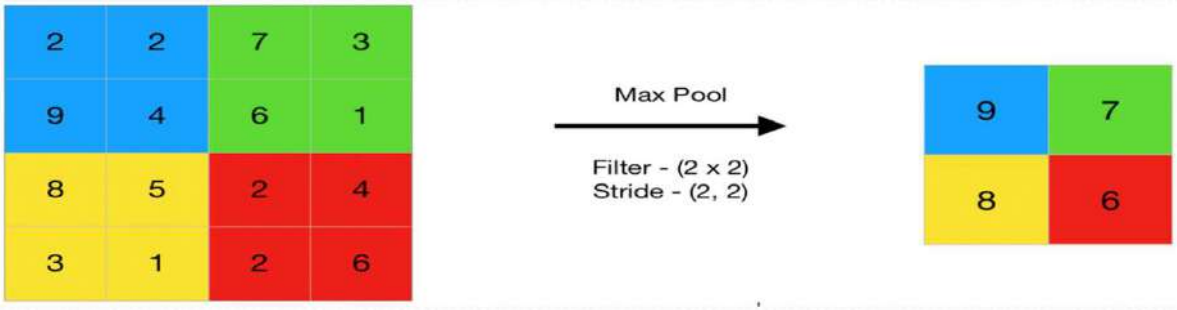
Let's perform pooling with a window size 2 and a stride 2

Pooling layer

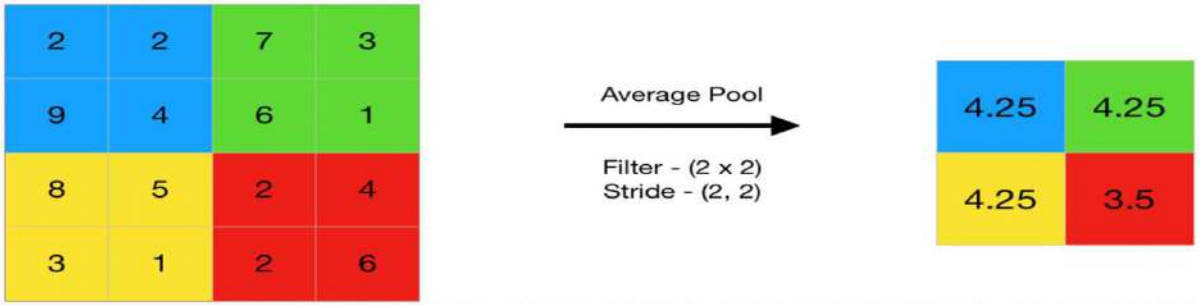
- Reduce the dimensionality of feature maps
- summarises the features present in a region of the feature map generated by a convolution layer.

Pooling layer

- Max pooling : selects the maximum element from the region of the feature map covered by the filter



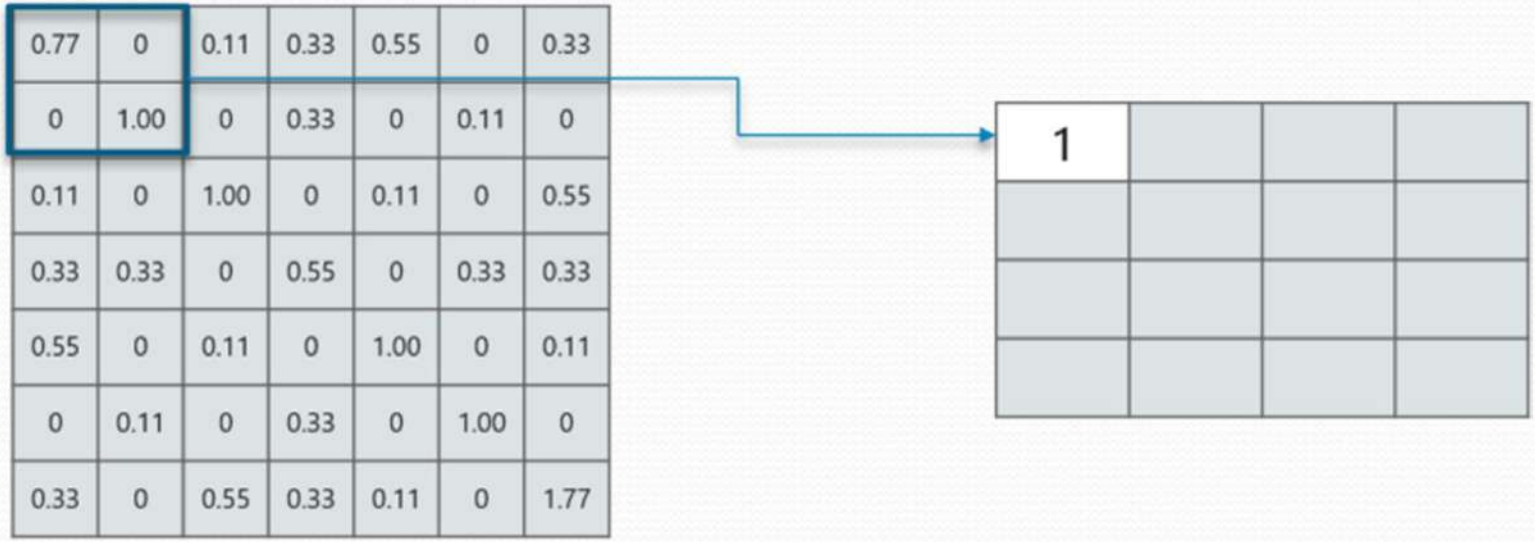
- Average pooling: computes the average of the elements present in the region of feature map covered by the filter



Pooling layer

- Pick a window size (usually 2 or 3)
- Pick a stride (usually 2)
- Move the window across your filtered images
- From each window, take the maximum value

Pooling layer



Applying max pooling

Pooling layer

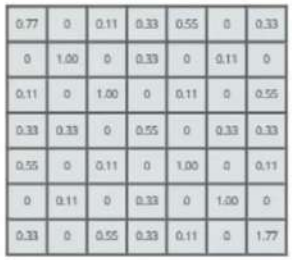
0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	1.77

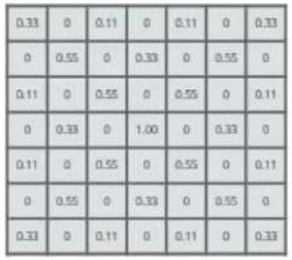
1.00	0.33	0.55	0.33
0.33	1.00	0.33	0.55
0.55	0.33	1.00	0.11
0.33	0.55	0.11	0.77

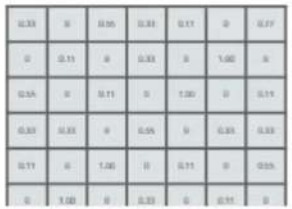
Applying max pooling

Do the same for other maps too ….

Output After Passing Through Pooling Layer







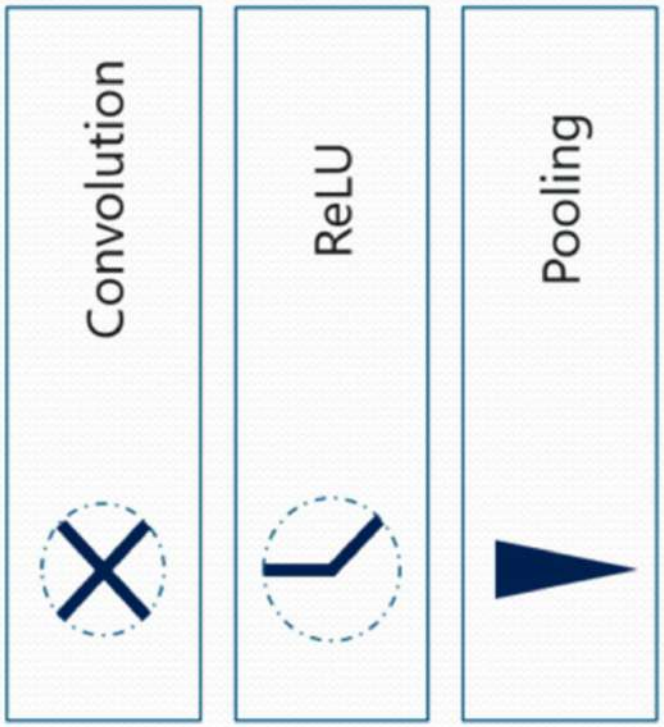
1.00	0.33	0.55	0.33
0.33	1.00	0.33	0.55
0.55	0.33	1.00	0.11
0.33	0.55	0.11	0.77

0.55	0.33	0.55	0.33
0.33	1.00	0.55	0.11
0.55	0.55	0.55	0.11
0.33	0.11	0.11	0.33

0.33	0.55	1.00	0.77
0.55	0.55	1.00	0.33
1.00	1.00	0.11	0.55
0.77	0.33	0.55	0.33

- from a 7×7 matrix after passing the input through 3 layers we get :

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1



1.00	0.33	0.55	0.33
0.33	1.00	0.33	0.55
0.55	0.33	1.00	0.11
0.33	0.55	0.11	0.77

0.55	0.33	0.55	0.33
0.33	1.00	0.55	0.11
0.55	0.55	0.55	0.11
0.33	0.11	0.11	0.33

0.33	0.55	1.00	0.77
0.55	0.55	1.00	0.33
1.00	1.00	0.11	0.55
0.77	0.33	0.55	0.33

- We can further reduce the image size by applying one more iteration of convolution, ReLU and pooling layers. Then we get

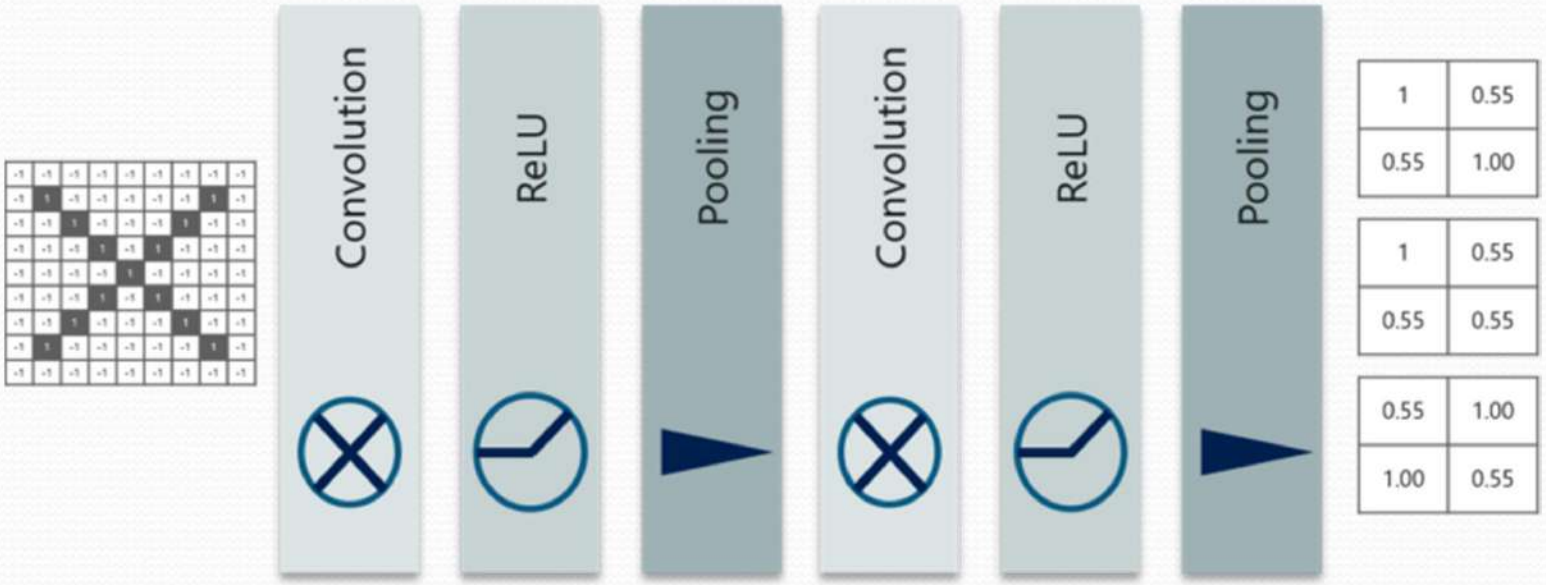
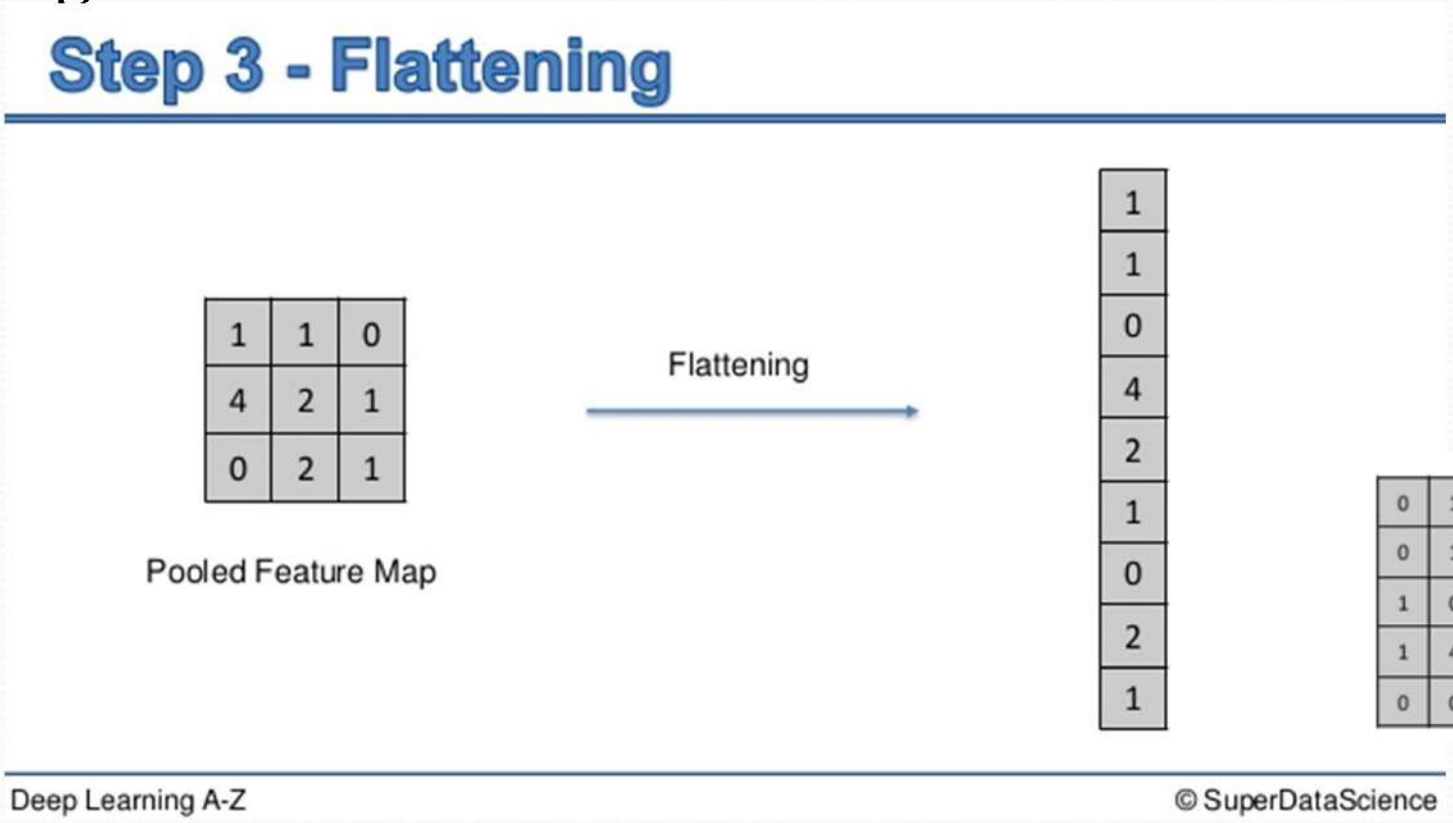


Fig: Stacking up the layers

Flattening

- Converts the 2D arrays from pooled feature maps into a single linear vector.



Flattening

- Feature extraction is now over.
- The next step is to perform classification using a fully connected layer.
- Before that perform flattening

1	0.55
0.55	1.00

1	0.55
0.55	0.55

0.55	1.00
1.00	0.55

1.00
0.55
0.55
1.00
1.00
0.55
0.55
0.55
0.55
1.00
1.00
0.55

Fully Connected Layer

This is the final layer where the actual classification happens

Here we take our filtered and shrinked images and put them into a single list

1	0.55
0.55	1.00

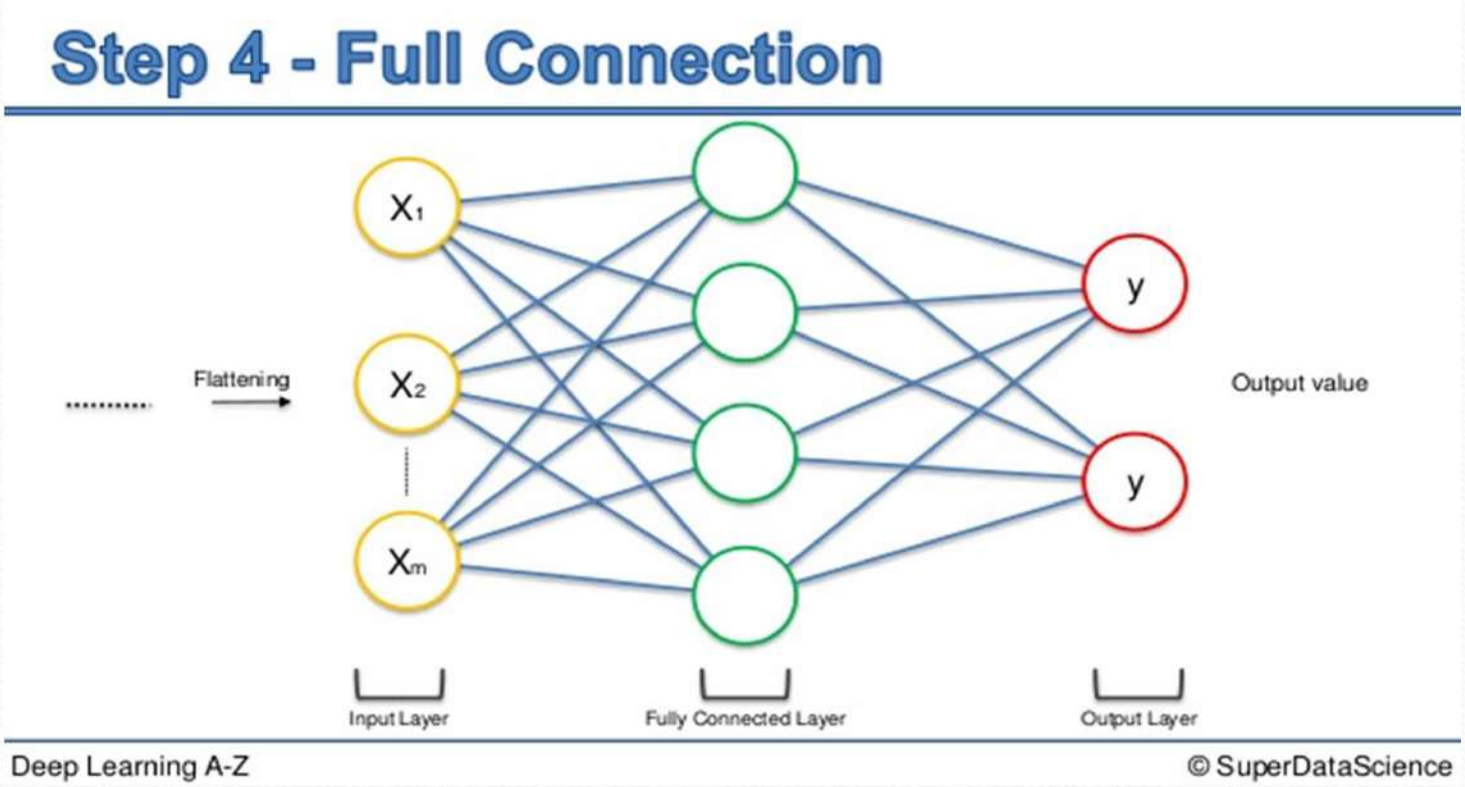
1	0.55
0.55	0.55

0.55	1.00
1.00	0.55

1.00
0.55
0.55
1.00
1.00
0.55
0.55
0.55
0.55
1.00
1.00
0.55

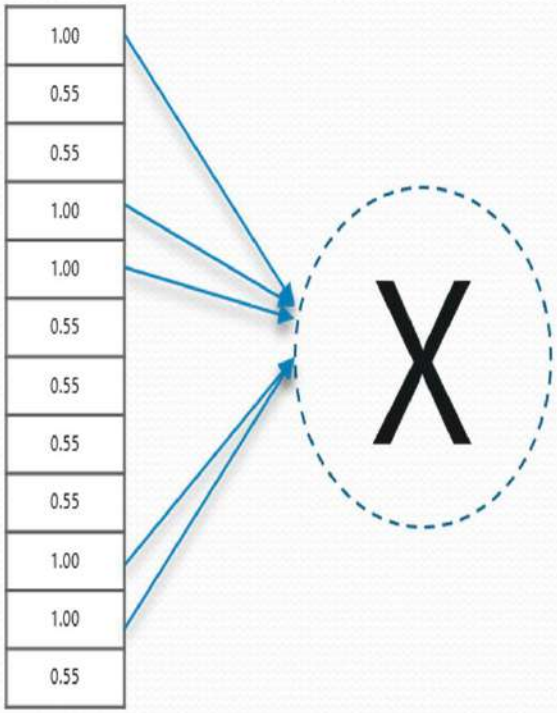
Fully connecetd layer

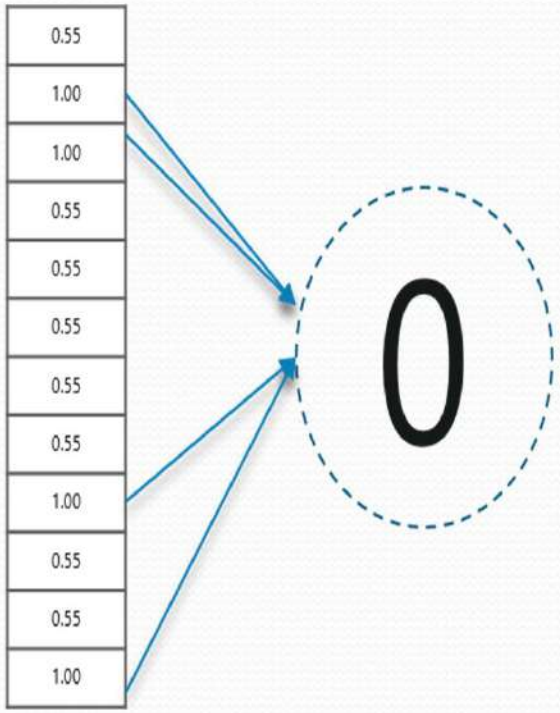
- Flattened vector is fed into a fully connected layer for classification.



Fully connected layer

- After passing through the fully connected layer, we get corresponding output vectors for both ‘X’ and ‘O’





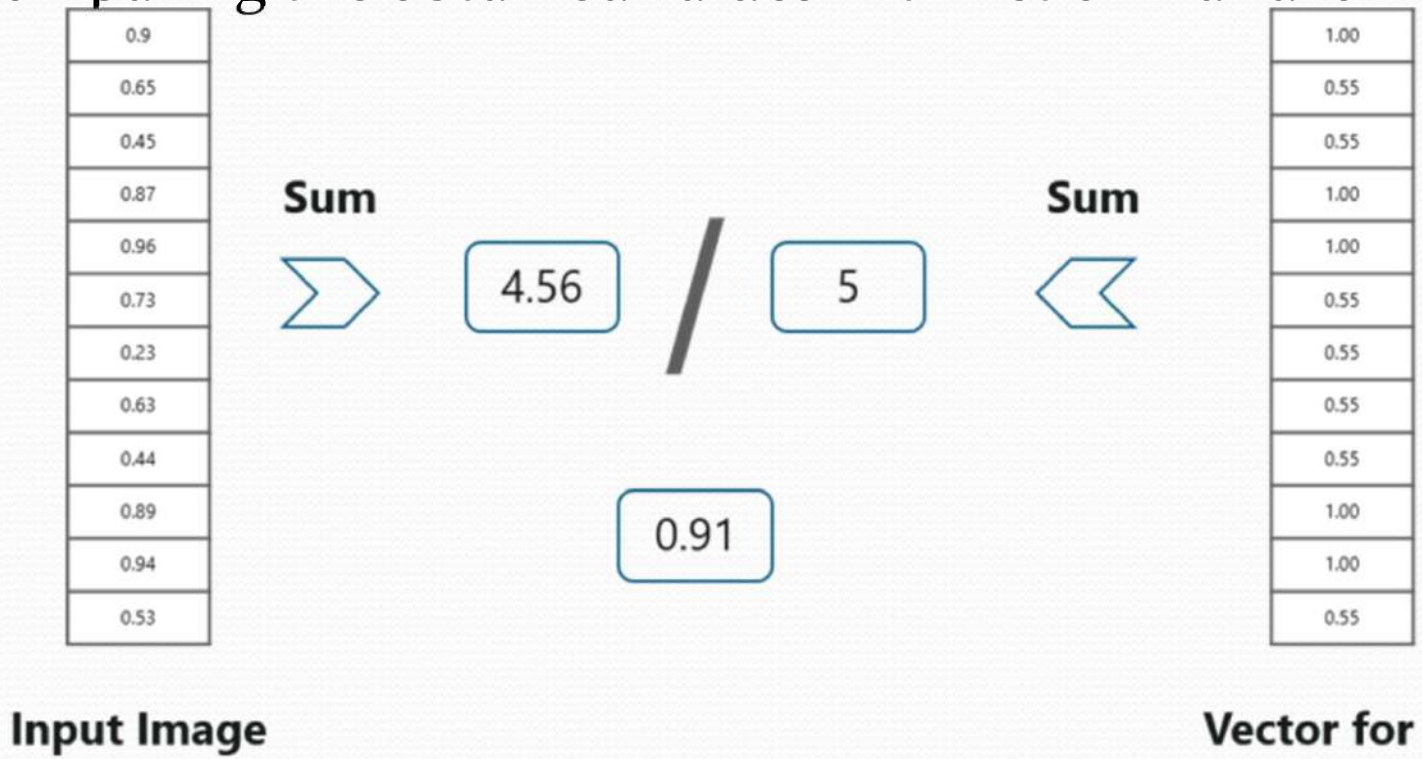
Prediction of an image

- Suppose we got the following output vector after passing through the CNN layers

0.9
0.65
0.45
0.87
0.96
0.73
0.23
0.63
0.44
0.89
0.94
0.53

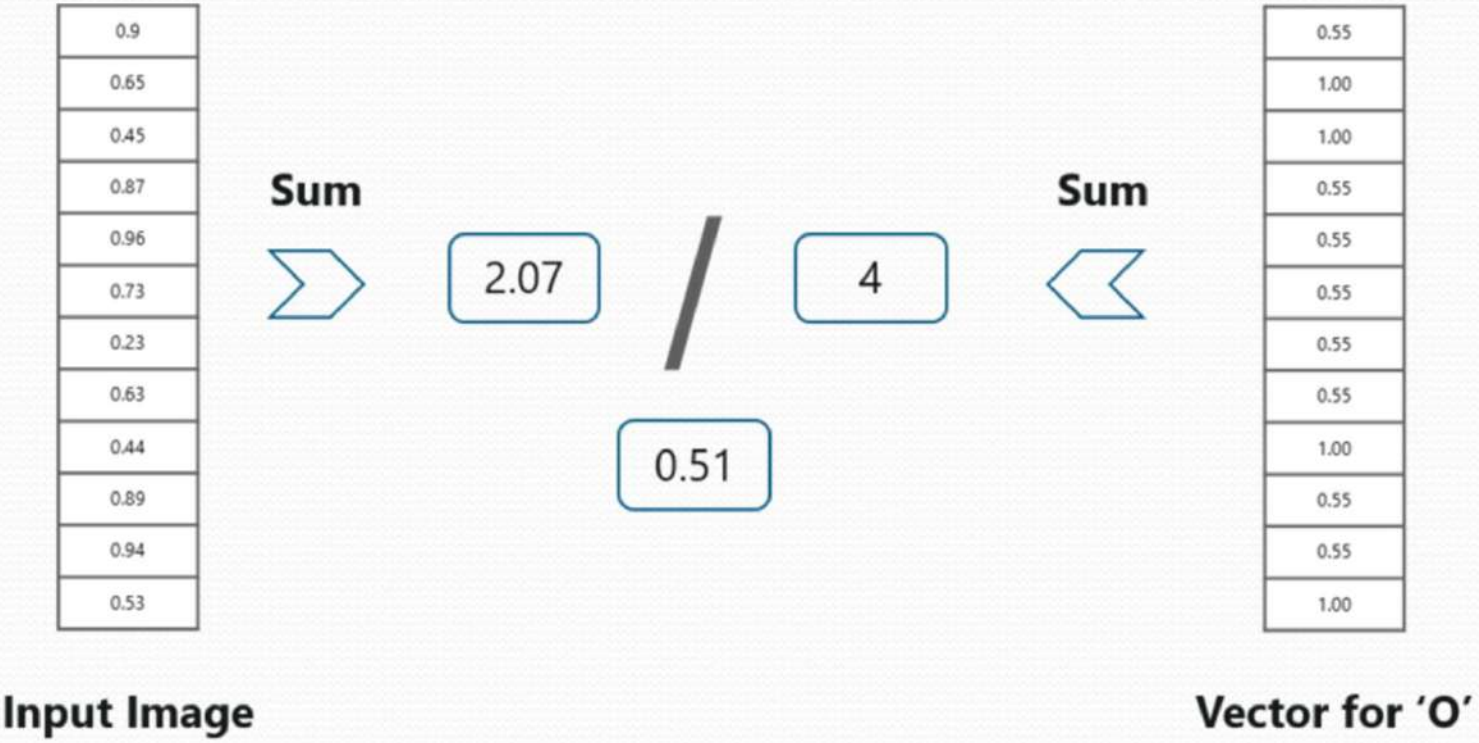
Prediction of an image

- We make predictions based on the output data by comparing the obtained values with list of ‘x’and ‘o’



Prediction of an image

- We make predictions based on the output data by comparing the obtained values with list of ‘x’and ‘o’



Prediction of an image

probability being 0.51 is less than 0.91

So we are now able to conclude that the resulting input image is an 'X'!